

Experiment 05

Aim: To apply navigation, routing and gestures in Flutter App.

Theory:

1. Navigation and Routing:

Flutter provides robust mechanisms for in-app navigation through both **named routes** and **direct route (MaterialPageRoute)** instantiation. Navigation in Flutter is managed by the **Navigator** widget, which maintains a stack-based history of route objects. The **Navigator.push()** and **Navigator.pop()** methods are used to transition between screens (routes) and to handle the back-stack.

- **Basic Navigation:** Using **Navigator.push(context, MaterialPageRoute(builder: (context) => SecondScreen()))** to navigate between widgets.

Named Routes: Defined in the **MaterialApp** widget via the **routes** parameter, allowing clean separation and easier route management, e.g.,

```
routes: {  
  
  '/': (context) => HomePage(),  
  
  '/login': (context) => LoginPage(),  
  
}
```

- This enables navigation using **Navigator.pushNamed(context, '/login')**.
- **Route Arguments:** Flutter allows passing data between screens using the constructor or **ModalRoute.of(context).settings.arguments**.

2. Gesture Detection

Flutter's gesture detection is powered by the **gesture arena** system. The **GestureDetector** widget plays a central role in listening to user touch events and mapping them to appropriate callbacks like **onTap**, **onDoubleTap**, **onLongPress**, **onPan**, and **onVerticalDrag**.

For more advanced gesture recognition, Flutter provides other widgets such as:

- **InkWell:** Material design widget that provides visual feedback (ripple effect) on taps.
- **Dismissible:** Handles swipe-to-dismiss behavior.

- **Draggable** and **DragTarget**: Used for building drag-and-drop interfaces.

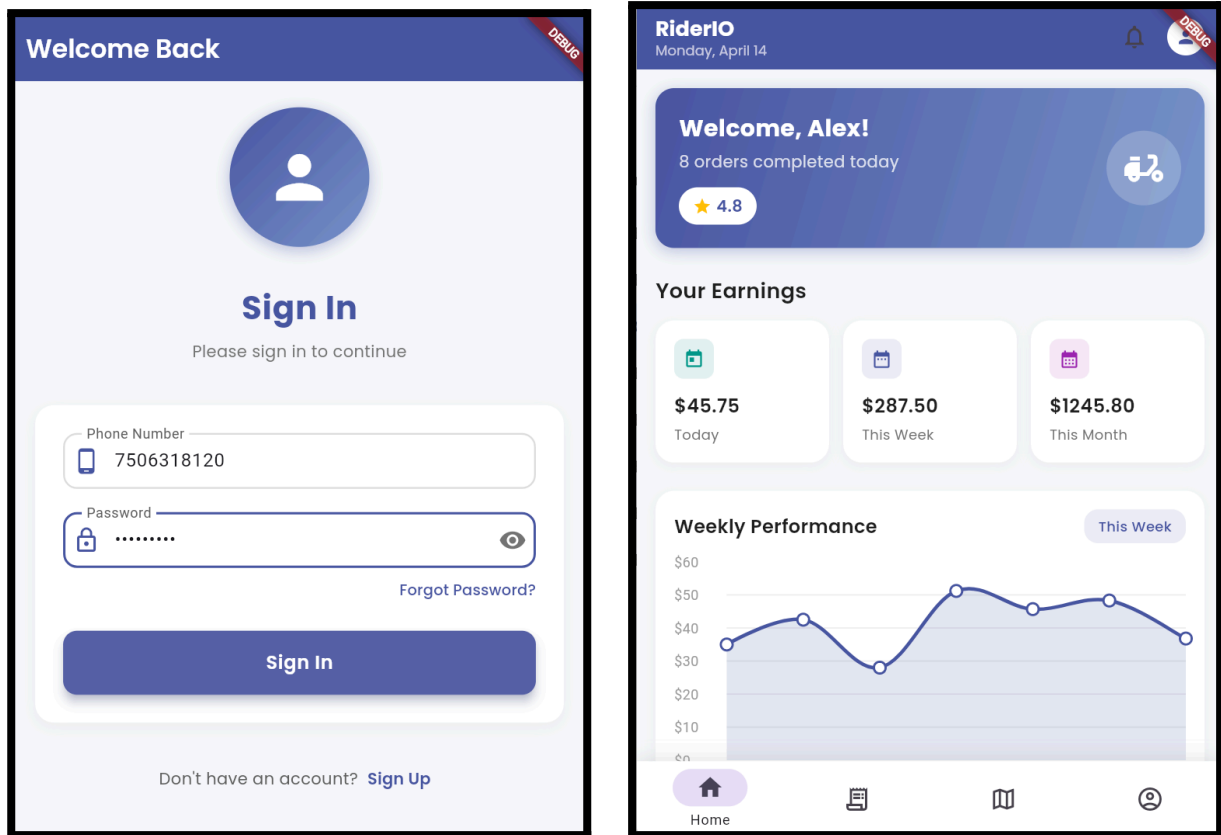
Implementation:

The **LoginScreen** is a beautifully animated and responsive Flutter widget designed to authenticate users using their phone number and password stored in Firestore. Upon initialization, it uses a fade-in animation to enhance the user experience. The screen consists of a top app bar with a welcoming message and a circular logo area with a user icon inside a gradient circle. Below that, the login form is displayed, consisting of two input fields: one for the phone number, validated to be exactly 10 digits, and another for the password with a toggle for visibility. A "Forgot Password?" link is also provided. The form is wrapped in a card-like container with subtle shadows for a clean, modern look. When the user presses the "Sign In" button, the app checks the Firestore database for matching credentials. If successful, it navigates to the **HomeScreen** with a smooth slide animation; otherwise, it shows an error message. There's also a bottom prompt allowing users to navigate to the **SignupScreen**. The design uses Google Fonts and consistent theming, giving it a polished and user-friendly interface.

Github Execution:

https://github.com/AdityaRaorane22/firebase_flutter/blob/main/lib/screens/login_screen.dart

https://github.com/AdityaRaorane22/firebase_flutter/blob/main/lib/screens/home_screen.dart

Output:**Conclusion:**

The LoginScreen efficiently combines a user-friendly interface, smooth transitions, and secure authentication with Firebase Firestore. The use of animations enhances the user experience, providing a polished look and feel. The form validation ensures data integrity, while error handling with snack bars provides clear feedback. The routing mechanism enables seamless navigation between the login screen and other sections of the app, such as the home and signup screens. Overall, the implementation of these features creates a secure, interactive, and aesthetically pleasing login flow for users.